NATIONAL RESEARCH UNIVERSITY
HIGHER SCHOOL OF ECONOMICS

Faculty of Computer Science
Bachelor's Programme "HSE and University of London Double Degree Programme in Data Science and Business Analytics"

Programming project report

on the topic Numerical Algorithms of Linear Algebra

Student:
group БПАД232 _____ З. Ю. Зинкин
Sign Name, Patronymic, Surname
4.02.2025
Date

Supervisor: Дмитрий Витальевич Трушин
Name, Patronymic, Surname
доцент / ФКН НИУ ВШЭ
Position / Company
4.02.2025 _____ _____
Date Grade Sign

Moscow 2025

Contents

Abstract

The main focus of the programming project is to study matrix decomposition algorithms from a theoretical point of view with its subsequent implementation on the C++ language.

1 Introduction

Decomposition algorithms are extensively used in Numerical Linear Algebra and Computer Science. They can help store large matrices, find eigenvalues of matrices, solve systems of linear equations and linear least squares problems and much more. This is why it is vital not only to recognize the potential use cases of decomposition algorithms, but also to understand the underlying theory and implementation of such algorithms.

The goal of my programming project is to learn modern algorithms of Numerical Linear Algebra with a focus on matrix decomposition algorithms, which represent a matrix in a certain way, as a product of other matrices, for further computation. In order to have a profound understanding of each algorithm, it is necessary to implement a C++ library from scratch.

C++ is the language of choice for the project, since it provides high performance through low-level memory control and compiler optimization. Moreover, its specialization in object-oriented design and template system make it suitable for working with matrices.

The library contains a class for matrix representation with implementation of matrix arithmetic and other necessary methods. Decomposition algorithms are implemented based on the aforementioned matrix class. The whole library is tested using Googletest[4].

Theoretical part(3) can help you understand the underlying theory of each of the algorithms. The technical part of the project is described in detail in the Library LinAlgTools(4) section of the paper. The results of the testing are documented in the Testing(5) section.

The information for the theoretical part of the project is taken from the books, which are cited at the end of the paper.

The result of the project is a C++ library `LinAlgTools`[9] with the following functionality:

- Matrix arithmetic
- QR Decomposition
- Bidiagonalization
- Singular Value Decomposition (SVD)

1.1 Motivation

Before proceeding with theoretical and technical details, it is essential to highlight the importance of this work and compare it to existing solutions. While the motivation for learning decomposition algorithms has already been outlined, the incentive behind the technical implementation must also be addressed. Let us first discuss existing libraries with similar functionality.

- *Eigen*[5] - a high-performance library, supporting dense and sparse matrices, solvers and many decomposition algorithms. However, due to prioritization of performance over readability, the code becomes unreadable. Moreover, the syntax of the library can be overwhelming.
- *Armadillo*[8] - a library, aiming towards a good balance between speed and ease of use. Decomposition algorithms are provided through integration with LAPACK[2], which relies on highly optimized, but opaque implementation. Therefore, the library's implementation may not be clear for an inexperienced user.
- *Dlib*[7] - a machine learning library with readable C++11 code and thorough documentation. However, its age makes it less appealing than Eigen (which updates regular) for professional use. Due to older C++ standard the library does not utilize powerful additions to the language, such as concepts, to drastically reduce the size of code. More importantly, the source code is flooded with inexpressive variable names, which make it difficult to follow.

As demonstrated, neither of the libraries are capable of providing clear and comprehensive implementation of the algorithms we need. They either prioritize performance at the expense of clarity or suffer from obsolete design choices. Therefore, `LinAlgTools` is developed as a learning tool, which can be used to properly understand each algorithms' implementation by any C++ amateur.

2 Description of functional and non-functional requirements for the program project

2.1 Functional requirements

- A *Matrix* class, which allows users to perform matrix arithmetic. The class must be able to support both real and complex elements. Moreover, the class must have a clear and intuitive interface.
- A *SubMatrix and ConstSubMatrix* classes for handling a particular part of an existing matrix without copying data. These classes must fully replicate the interface of the base *Matrix* class to ensure complete compatibility. Furthermore, cross-type operations must be implemented to perform arithmetic between objects of *Matrix* and *SubMatrix* classes of appropriate sizes.
- Implementation of numerically stable algorithms:
  - QR Decomposition - representation of a matrix as a product of Q and R matrices, where Q is unitary and R is upper-triangular. The decomposition must be implemented using both Householder reflections and Givens rotations.
  - Bidiagonalization - representation of a matrix as a product of U, B and V^* matrices, where U, V are unitary and B is bidiagonal.
  - SVD Decomposition - representation of a matrix as a product of U, Σ and V^* matrices, where U, V are unitary and Σ is a diagonal matrix.
- Extensive testing must be done to guarantee proper functioning of the library. Randomized performance tests are to be analyzed.

2.2 Non-functional requirements

- Implementation of the library on the C++ language without third-party libraries, except for Googletest.
- *Git*, a version control system, and a remote repository on *GitHub* [9].
- *Clang* compiler with C++23 standard support.
- Code formatter *clang-format* to automatically format code based on the Microsoft style with additional changes.
- Static code analyzer *clang-tidy* to identify potential errors and to stick to uniform naming of variables, functions, etc.
- A third-party library *Googletest* [4] for writing tests.
- A text editor with IDE capabilities *Neovim*.
- Minimum RAM requirement: 4GB.

3 Theoretical part

Theoretical part is divided into two parts: *Definitions* and *Algorithms*. The first part establishes rigorous framework for the algorithm discussed later. The second part discusses implemented algorithms from a theoretical point of view by providing verbal description, mathematical statements with proofs and pseudocode. Special emphasis is made on proofs, as they not only mathematically validate the algorithms, but often reveal their structure. Pseudocode sections deliberately omit edge-cases to focus on core ideas and maintain clarity.

3.1 Main definitions

Definition 1. An $m \times n$ *matrix* over a field $\mathbb{F}$ (either $\mathbb{C}$ or $\mathbb{R}$) is a rectangular array of elements of $\mathbb{F}$ arranged in m rows and n columns:

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$$

Remark. A set of all $m \times n$ matrices over a field $\mathbb{F}$ we denote $\mathbb{F}^{m \times n}$.

Definition 2. A matrix $A \in \mathbb{F}^{m \times n}$ is called *diagonal* if all its elements not on the diagonal are 0:

$$\begin{bmatrix} a_{11} & 0 & 0 & \dots & 0 & \dots & 0 \\ 0 & a_{22} & 0 & \dots & 0 & \dots & 0 \\ 0 & 0 & a_{33} & \dots & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & a_{mk} & \dots & 0 \end{bmatrix}$$

Remark. A special case of a diagonal matrix is an $n \times n$ *Identity* matrix:

$$I_n = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 \end{bmatrix}$$

Definition 3. A matrix $A \in \mathbb{F}^{m \times n}$ is called *upper triangular* if all its elements below the diagonal are 0:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1k} & \dots & a_{1n} \\ 0 & a_{22} & a_{23} & \dots & a_{2k} & \dots & a_{2n} \\ 0 & 0 & a_{33} & \dots & a_{3k} & \dots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & a_{mk} & \dots & a_{mn} \end{bmatrix}$$

Definition 4. The superdiagonal of a matrix is the set of elements directly above the elements comprising the diagonal.

Definition 5. A matrix $A \in \mathbb{F}^{m \times n}$ is called *bidiagonal* if all its elements below the diagonal and above the superdiagonal are 0:

$$\begin{bmatrix} a_{11} & a_{12} & 0 & \dots & 0 & 0 & \dots & 0 \\ 0 & a_{22} & a_{23} & \dots & 0 & 0 & \dots & 0 \\ 0 & 0 & a_{33} & \dots & 0 & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & a_{mk} & a_{mk+1} & \dots & 0 \end{bmatrix}$$

Definition 6. A vector is an $1 \times n$ or $n \times 1$ matrix:

$$\begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} \text{ or } \begin{bmatrix} a_1 & a_2 & \dots & a_n \end{bmatrix}$$

Remark. A set of all $m \times 1$ or $1 \times m$ vectors over $\mathbb{F}$ we denote $\mathbb{F}^m$.

Definition 7. A *Hermitian conjugate* of a matrix $A \in \mathbb{C}^{m \times n}$ is an $n \times m$ matrix, obtained by transposing A and applying complex conjugate to each of entry: $(A^*)_{ij} = \overline{A}_{ji}$.

Remark. For real matrices *Hermitian conjugate* coincides with transposition of the matrix: $A^* = A^T$.

Definition 8. A matrix $A \in \mathbb{C}^{m \times n}$ is called *Hermitian* if $A^* = A$.

Definition 9. A matrix $A \in \mathbb{C}^{m \times n}$ is called *Unitary* if $A^* = A^{-1}$. As a consequence, $A^*A = AA^* = I$.

Definition 10. A matrix $A \in \mathbb{R}^{m \times n}$ is called *Orthogonal* if $A^T = A^{-1}$. As a consequence, $A^TA = AA^T = I$.

Statement 1. *A product of unitary (orthogonal) matrices is also unitary (orthogonal).*

Proof. Let U and V be unitary:

$$(UV)^*UV = V^*U^*UV = V^*V = I$$

□

Definition 11. A *scalar product* is a bilinear form $\langle \cdot, \cdot \rangle : \mathbb{F}^n \times \mathbb{F}^m \to \mathbb{F}$. In a Euclidean space, such as $\mathbb{R}^m$ or $\mathbb{C}^m$ the scalar product of two vector $v, w \in \mathbb{C}^m$ is defined in the following way:

$$\langle v, w \rangle = w^*v = \sum_{i=1}^{m} v_i \overline{w_i}$$

Definition 12. A 2-norm, or Euclidean norm, on an Euclidean space is a function: $\|\cdot\|_2 : \mathbb{F}^n \to \mathbb{R}_+$.
Let $v \in \mathbb{F}^n$:

$$\|v\|_2 = \sqrt{\langle v, v \rangle} = \sqrt{\sum_{i=1}^{m} |v_i|^2}$$

Remark. Geometrically, a 2-norm returns the distance from the origin to the point in the space.

Definition 13. A matrix is called *sparse* if the majority of its elements are 0.

Definition 14. A matrix is called *dense* if the majority of its elements are non-zero.

Remark. There is no definite rule to tell whether the matrix is sparse or dense, but the usual practice is to consider a matrix sparse if the number of non-zero elements approximately equals to the number of rows or columns.

Definition 15. The i-th standard basis vector of a vector space $\mathbb{F}^m$ will be denoted $e_i = [0, \dots \underset{i}{1}, \dots, 0]$

3.2 Algorithms

3.2.1 Householder reflection

Definition 16. A *Householder reflection* is a linear transformation which reflects a vector about a hyperplane[3].

Given a vector $v \in \mathbb{F}^m : \|v\| = 1$, an $m \times m$ matrix P (or more commonly $H(v)$) of the form

$$P = H(v) = I - 2vv^*$$

is called a *Householder reflector* or *Householder matrix*. The matrix allows us to reflect vector along the hyperplane, spanned by v. Moreover, a Householder matrix is unitary and hermitian ($H^{-1} = H^* = H$), which turns out to be key in the QR decomposition algorithm.

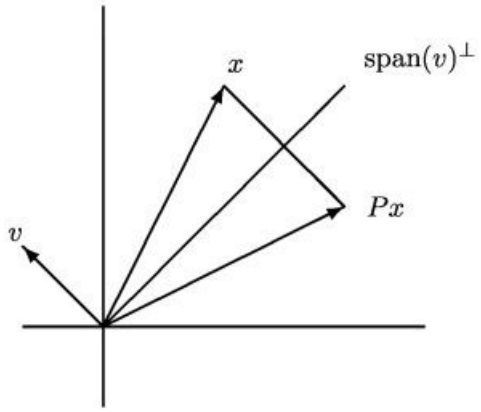

Figure 1: Householder reflection applied to a vector x

Statement 2. *$\forall a, b \in \mathbb{C}^n : \|a\|_2 = \|b\|_2$, $\exists \gamma \in \mathbb{C} : |\gamma| = 1$ and $\exists v \in \mathbb{C}^n : \|v\|_2 = 1$, such that $H(v)a = \gamma b$.*

Proof. By direct computation we can find the required γ and v:

$$H(v) = I - 2vv^*$$

$$H(v)a = a - 2vv^*a = \gamma b$$

$$2(v^*a)v = a - \gamma b \tag{1}$$

From here it follows that v must be of the form:

$$v = \mu \frac{a - \gamma b}{\|a - \gamma b\|_2}$$

Since we are proving the existence, we can set μ to 1. Therefore

$$v = \frac{a - \gamma b}{\|a - \gamma b\|_2}$$

Plugging it into (1):

$$2\frac{(a - \gamma b)^*}{\|a - \gamma b\|_2} a \frac{a - \gamma b}{\|a - \gamma b\|_2} = a - \gamma b$$

$$2(a - \gamma b)^* a = \|a - \gamma b\|_2^2$$

$$2(a - \gamma b)^* a = (a - \gamma b)^*(a - \gamma b)$$

$$2a^*a - 2\overline{\gamma} b^* a = a^*a + b^*b - \gamma a^* b - \overline{\gamma} b^* a$$

$$\overline{\gamma} b^* a = \mathrm{Re}(\overline{\gamma} b^* a)$$

$$\gamma = \pm \frac{b^* a}{|b^* a|}$$

If $a \perp b$, γ can be any number, such that $|\gamma| = 1$. If $a \parallel b$, then we should carefully choose the sign to guarantee numeric stability of the transformation. □

Corollary 1. $\forall x \in \mathbb{C}^n \; \exists v \in \mathbb{C}^n : \|v\|_2 = 1$ *and*

$$H(v)x = \gamma \begin{bmatrix} \|x\|_2 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

$$v = \frac{x - \gamma \|x_2\|_2 e_1}{\|x - \gamma \|x\|_2 e_1\|_2}, \; \gamma = -\frac{x_1}{|x_1|}$$

The corollary yields an algorithm for transforming a vector, and if we want to apply the transformation to a matrix from the left (right), we can use:

Algorithm 1 Householder Left Reflection

```
function HouseholderLeftReflection(A ∈ ℂ^{m×n}, x ∈ ℂ^m)
    v = x
    v_1 = x_1 − sign(x_1) · ‖x‖_2
    v = v · 1/‖v‖
    M = A − (2v) · (v* A)
    return M
end function
```

3.2.2 Givens rotations

Definition 17. A *Givens rotation* is a linear transformation which amounts to a counterclockwise rotation by an angle of θ[3].

In case we need to zero specific elements of the matrix, *Givens rotations* are our tool of choice. The matrix of a *Givens rotation* is of the form ($c = \cos\theta$, $s = \sin\theta$):

$$G(i,j,\theta) = \begin{bmatrix} 1 & \cdots & 0 & \cdots & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & & \vdots & & \vdots \\ 0 & \cdots & c & \cdots & -s & \cdots & 0 \\ \vdots & & \vdots & \ddots & \vdots & & \vdots \\ 0 & \cdots & s & \cdots & c & \cdots & 0 \\ \vdots & & \vdots & & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & \cdots & 0 & \cdots & 1 \end{bmatrix} \begin{matrix} \\ \\ i \\ \\ j \\ \\ \\ \end{matrix}$$
$$\begin{matrix} & & i & & j & & \end{matrix}$$

Statement 3. $\forall v \in \mathbb{F}^m, i, j \in \mathbb{N}_+ \; \exists \theta \in \mathbb{F}$:

$$G(i,j,\theta) \begin{bmatrix} \vdots \\ a \\ \vdots \\ b \\ \vdots \end{bmatrix} = \begin{bmatrix} \vdots \\ r \\ \vdots \\ 0 \\ \vdots \end{bmatrix} \begin{matrix} \\ i \\ \\ \\ j \\ \\ \end{matrix}$$

Proof. If $x \in \mathbb{F}^m$ and

$$y = G(i,j,\theta)x$$

Then:

$$y_j = \begin{cases} cx_i - sx_k, & j = i \\ sx_i + cx_k, & j = k \\ x_j, & j \neq i,k \end{cases}$$

From here we can deduct that $y_k = 0$ if:

$$c = \frac{|x_i|}{\sqrt{|x_i|^2 + |x_k|^2}}, \quad s = \frac{-\overline{x}_k}{\sqrt{|x_i|^2 + |x_k|^2}} \cdot \frac{x_i}{|x_i|}$$

□

Notice that G is almost the identity matrix. We can exploit this structure to achieve faster computation by apply the transformation directly to the rows which are affected by G.

Algorithm 2 Givens Left Rotation

function GivensLeftRotation($A \in \mathbb{F}^{m\times n}$, $i \in \mathbb{N}_+$, $j \in \mathbb{N}_+$, $a \in \mathbb{F}$, $b \in \mathbb{F}$)
  $r = \sqrt{|a|^2 + |b|^2}$
  $c = a/r$
  $s = -b/r$
  $M = A$
  for k = 1 to n **do**
    $M_{ik} = \overline{c} \cdot A_{ik} - \overline{s} \cdot A_{jk}$
    $M_{jk} = s \cdot A_{ik} + c \cdot A_{jk}$
  end for
  return M
end function

3.2.3 QR decomposition

QR decomposition is a tool for solving systems of linear equations and finding eigenvalues of a matrix. The latter is the main reason for its implementation, since it allows to use *QR Algorithm* which will be discussed later. The decomposition stability together with its use in other algorithms make it indispensable in our case.

Theorem 1. *For any matrix $A \in \mathbb{F}^{m\times n}$ exist a unitary matrix $Q \in \mathbb{F}^{m\times m}$ and an upper-triangular matrix $R \in \mathbb{F}^{m\times n}$, such that $A = QR$.*

Proof. Using Householder rotations (for convenience $A \in \mathbb{F}^{4\times 3}$):

$$A = \begin{bmatrix} \star & \star & \star \\ \star & \star & \star \\ \star & \star & \star \\ \star & \star & \star \end{bmatrix} \overset{H(v_1)}{\to} \begin{bmatrix} \star & \star & \star \\ 0 & \star & \star \\ 0 & \star & \star \\ 0 & \star & \star \end{bmatrix} \overset{H(v_2)}{\to} \begin{bmatrix} \star & \star & \star \\ 0 & \star & \star \\ 0 & 0 & \star \\ 0 & 0 & \star \end{bmatrix} \overset{H(v_3)}{\to} \begin{bmatrix} \star & \star & \star \\ 0 & \star & \star \\ 0 & 0 & \star \\ 0 & 0 & 0 \end{bmatrix} = R$$

$$H_3 H_2 H_1 A = R$$

$$A = H_1^{-1} H_2^{-1} H_3^{-1} R = H_1 H_2 H_3 R = QR$$

□

Algorithm 3 Householder QR Decomposition

function HouseholderQR($A \in \mathbb{F}^{m\times n}$)
Complexity: $\mathcal{O}(4mn^2 - \frac{4}{3}n^3)$
  $Q = I_m$
  $R = A$
  for $k = 1$ to n **do**
    $\mathbf{x} = R[k : m, k]$
    v = HouseholderReduction(x)
    $R[k : m, k : n]$ = HouseholderLeftReflection($R[k : m, k : n], v$)
    $Q[k : m, 1 : m]$ = HouseholderLeftReflection($Q[k : m, 1 : m], v$)
  end for
  $Q = Q^T$
  return Q, R
end function

Proof. Using Givens rotations (for convenience $A \in \mathbb{F}^{3\times 2}$):

$$A = \begin{bmatrix} \star & \star \\ \star & \star \\ \star & \star \end{bmatrix} \overset{G_{13}}{\to} \begin{bmatrix} \star & \star \\ \star & \star \\ 0 & \star \end{bmatrix} \overset{G_{12}}{\to} \begin{bmatrix} \star & \star \\ 0 & \star \\ 0 & \star \end{bmatrix} \overset{G_{23}}{\to} \begin{bmatrix} \star & \star \\ 0 & \star \\ 0 & 0 \end{bmatrix} = R$$

$$G_{23}G_{12}G_{13}A = R$$

$$A = G_{13}^* G_{12}^* G_{23}^* R = QR$$

□

Algorithm 4 Givens QR Decomposition

```
function GivensQR(A ∈ F^{m×n})
Complexity: O(6mn^2 − (5/2)n^3)
    Q = I_m
    R = A
    for k = 1 to n do
        for i = m downto k + 1 do
            a = R[i − 1, j]
            b = R[i, j]
            if b ≠ 0 then
                R[1 : m, k : n] = GivensLeftRotation(R[1 : m, k : n], i − 1, i, a, b)
                Q[1 : m, 1 : m] = GivensRightRotation(Q[1 : m, 1 : m], i − 1, i, a, b)
            end if
        end for
    end for
    return Q, R
end function
```

While Householder implementation might look superior to Givens due to better complexity, it is not always the case. The main strength of Givens rotations is the ability to zero out specific elements, making it particularly good for sparse matrices or matrices, which are close to being upper-triangular. More information on sparse orthogonal decomposition can be found in this paper [1]. Householder reflections, on the other hand, perform better on dense matrices due to full-column operations.

Therefore each implementation has its own strengths and weaknesses, which should be taken into account when working with structured matrices.

4 Library LinAlgTools

The library is designed with a modular and extensible architecture to efficiently handle decomposition algorithms. It has the following architecture:

```
LinAlgTools/
└── src/
    ├── types/
    │   ├── matrix.h ..................... Primary matrix
    │   ├── sub_matrix.h ..................... Mutable view
    │   └── const_sub_matrix.h ..................... Immutable view
    ├── core/
    │   ├── indices.h ..................... Index/slice types
    │   ├── is_complex.h ..................... Complex number type traits
    │   ├── math_utils.h ..................... Math helpers
    │   └── matrix_traits.h ..................... Type traits
    └── algorithms/
        ├── givens.h ..................... Givens rotations
        ├── householder.h ..................... Householder reflections
        └── qr_decomposition.h ..................... QR decomposition algorithms
```

Figure 2: Directory structure of the `LinAlgTools` library

4.1 Types

This part of the library contains the `Matrix` class and its derivatives: `SubMatrix` and `ConstSubMatrix` classes.

- The `Matrix` class allows to construct matrices of different sizes, with elements of different types (C++ fundamental types or `std::complex`) and perform matrix arithmetic. The class features all necessary methods for algorithms to work correctly, as well as many others that may be useful in basic linear algebra calculations. Moreover, `concepts` are used to allow for cross-type matrix arithmetic. Hence, it is possible to multiply/add/subtract matrices of different types, for example, an object of a `matrix` class and an object of a `submatrix` class.

```
1  LinAlgTools::Matrix<int> matrix = {{1, 2}, {3, 4}};
2  std::cout << matrix; //[[1, 2], [3, 4]]
3
4  LinAlgTools::Matrix<int> sq_matrix = {2};
5  std::cout << sq_matrix; //[[0, 0], [0, 0]]
6
7  LinAlgTools::Matrix<int> identity = LinAlgTools::Matrix<int>::Identity(2);
8  std::cout << identity; //[[1, 0], [0, 1]]
```

- The `SubMatrix` and `ConstSubMatrix` classes allow to perform operations with a particular part of the matrix without copying data. This is particularly useful in our case, since we want to apply transformations to a submatrix, which can greatly reduce the number of unnecessary computations. The classes also allow to create an object of a `matrix` class.

```
1  LinAlgTools::Matrix<int> matrix = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
2  LinAlgTools::SubMatrix<int> submatrix = {matrix, {0, 1}, {0, 1}};
3  std::cout << submatrix; //[[1, 2], [4, 5]]1
4
5  submatrix *= 5;
6  std::cout << matrix; //[[5, 10, 3], [20, 25, 6], [7, 8, 9]];
```

4.2 Core

The entirety of this part of the library is dedicated towards providing useful functions or other functionality, which is used throughout the project.

- `Math_utils` provide the ability to calculate the sign of a real or a complex number, check whether a matrix is diagonal, orthogonal and much more. `Indices` store uniform types of matrix entry index, row and column slices. This allows to easily change the required type if needed, and the changes will take place throughout the whole library.

```
1   LinAlgTools::Matrix<long double> matrix = {{1e-16, 1e-16}};
2   LinAlgTools::Matrix<long double> zero_matrix = {{0, 0}};
3   std::cout << LinAlgTools::AreEqualMatrices(matrix, zero_matrix); //true
```

- `Matrix_traits` and `is_complex` are used to handle different matrix types and complex number respectively. Concepts, mentioned earlier, were introduced in C++ and are a tool which allows to have uniform template for any matrix class.

```
1   template<Core::MatrixType F, Core::MatrixType S>
2 Matrix<typename F::ElementType> operator+(const F& lhs, const S& rhs) {
3   \*Implementation*\
4 }
```

4.3 Algorithms

This part of the library contains the implementation of the algorithms, which were extensively discussed above. Majority of algorithms' subroutines are implemented as separate functions and can be used as linear transformations. Moreover, the implementation of the algorithms exactly matches its pseudocode counterpart for preserving cohesion and level of abstraction.

- `Householder` contains `HouseholderVectorReduction` for transforming a vector, zeroing out each elements below the first first one. `HouseholderLeftReflection` and `HouseholderRightReflection`, provided a vector, are used to apply a reflection to a matrix.
- `Givens` contains `GetGivensPair`, which calculates coefficients for further rotation. `GivensLeftRotation` and `GivensRightRotation`, provided a vector, are used to apply a rotation to a matrix.
- `QR_Decomposition` contains two implementations of the decomposition: one based on Householder reflections and another based on Givens rotations. The former utilizes `HouseholderVectorReduction` and `HouseholderLeftReflection` to transform the matrix to the required form. Reflections are calculated in-place and to a particular submatrix to avoid unnecessary computations and copies. The latter uses `GetGivensCoefficients`, `GivensLeftRotation` and `GivensRightRotation` to calculate matrices Q and R. Left rotations are applied to submatrices of R, while right rotations are applied to submatrices of Q.

```
1     LinAlgTools::Matrix<int> matrix = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
2     auto [Q_1, R_1] = LinAlgTools::Algorithm::HouseholderQR(matrix);
3     auto [Q_2, R_2] = LinAlgTools::Algorithm::GivensQR(matrix);
```

5 Testing

The library utilizes a testing framework to validate correctness and measure performance. The test architecture is organized as follows:

```
LinAlgTools/
└── tests/
    ├── benchmark_tests/
    │   ├── benchmark_utils/
    │   │   ├── algorithm_benchmark.h
    │   │   └── random_generator.h
    │   └── qr_performance_tests.cpp
    └── validation_tests/
        ├── matrix_tests.cpp
        ├── sub_matrix_tests.cpp
        ├── const_sub_matrix_tests.cpp
        └── qr_tests.cpp
```

Figure 3: Directory structure of the `Tests` section

5.1 Validation tests

Each implemented function, class and algorithm in the library is tested on multiple test-cases. Proper functioning of the library is ensured via Googletest [4].

All tests related to matrices guarantee that the implementation works as intended. Particular attention is paid to edge-cases. For the QR Decomposition algorithm each tests validates the required properties of matrices Q and R (Q must be unitary, while R must be upper-triangular). Afterwards, reconstructed matrix is compared to the original one, ensuring that the error stays within the range of machine error. The same tests are ran twice: first for Householder implementation, then for Givens. Similarly, SVD Decomposition is tested to ensure that matrices U and V^* are unitary, and Σ is diagonal. Correct reconstruction is also tested.

5.2 Benchmark tests

Both *QR Decomposition* algorithms and *SVD Decomposition* are benchmarked on random matrices. For each matrix size each algorithm calculates the respective decomposition on 10 dense matrices, then stores the information about the size of the matrix, mean time, minimum time, maximum time, standard deviation in a `.csv` file `qr_performance_{algorithm}.csv`

Remark. The benchmark tests have been conducted on a MacBook Pro with Apple M2 pro chip and 32gb of memory.

5.2.1 Householder vs. Givens

The same seed is sent to the random generator to ensure the correctness of the comparison. Performance is measured on dense matrices.

I have mentioned before that this is the perfect setting for Householder implementation, and data below also proves this point. We can see that as matrix size increases, mean execution of Givens implementation (figure 4) exceeds that of Householder one. This is to be expected due to the increased complexity. On figure 5 for each matrix size I calculate the ratio of Givens mean time and Householder mean time. Therefore, it also demonstrates that Givens' performance is worse than Householders'. However, we can notice that the difference in performance decreases, as the size increases. Yet, it is impossible to conclude whether this effect will still hold on larger matrices due to small sample size.

In the theoretical part I have also stated that Givens implementations is better for sparse matrices. Unfortunately, the testing I have performed did not empirically prove it, that is why I do not present this data. My implementation of the Givens algorithm does not properly exploit the sparse structure of a matrix. Therefore, more optimization is required to see a difference, which is provided, for example, in *Numpy* [6]. Since such optimization was not a part of a project, it is not implemented in LinAlgTools. However, available scalability of the project allows for the realization of better optimization for structured matrices.

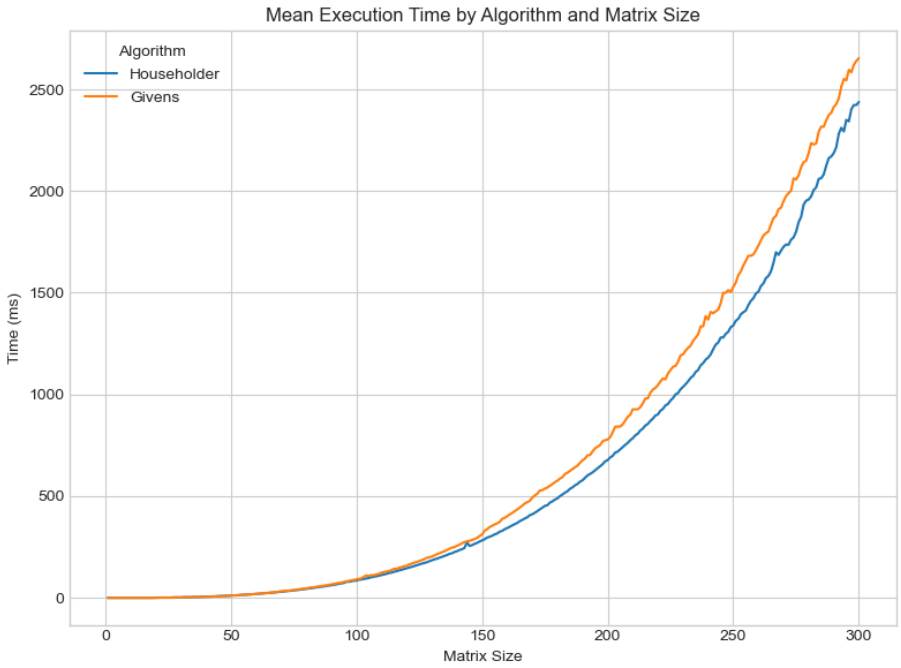

Figure 4: Mean execution time by algorithm and matrix size

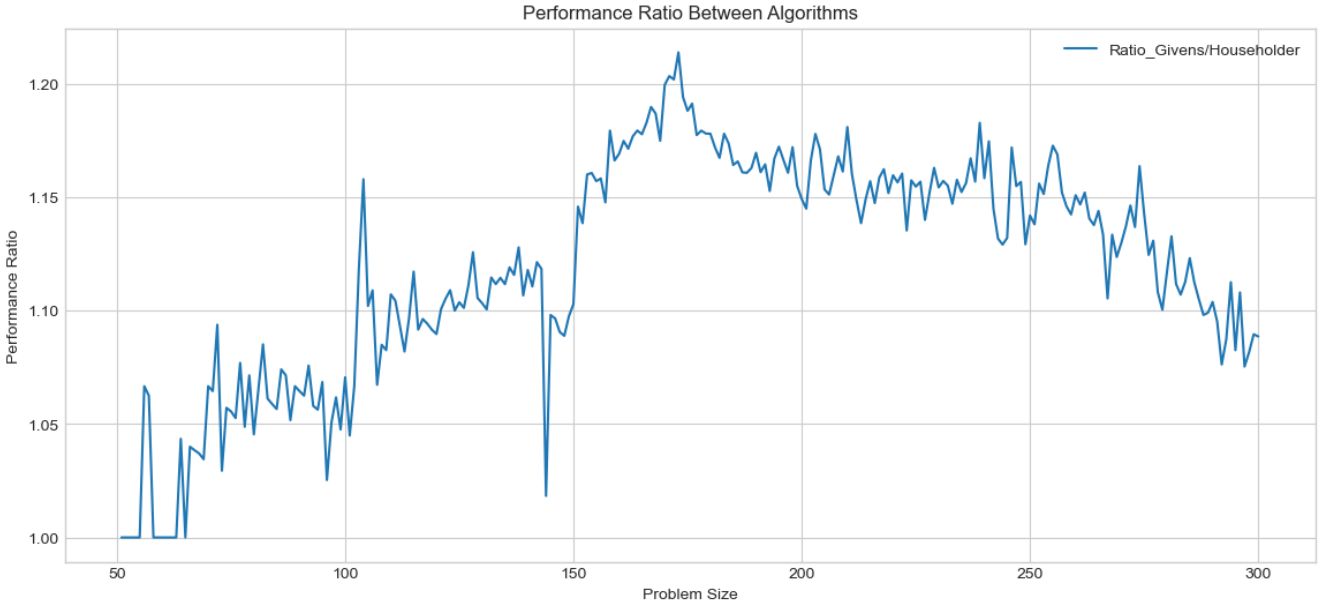

Figure 5: Standard deviation of execution times by algorithm

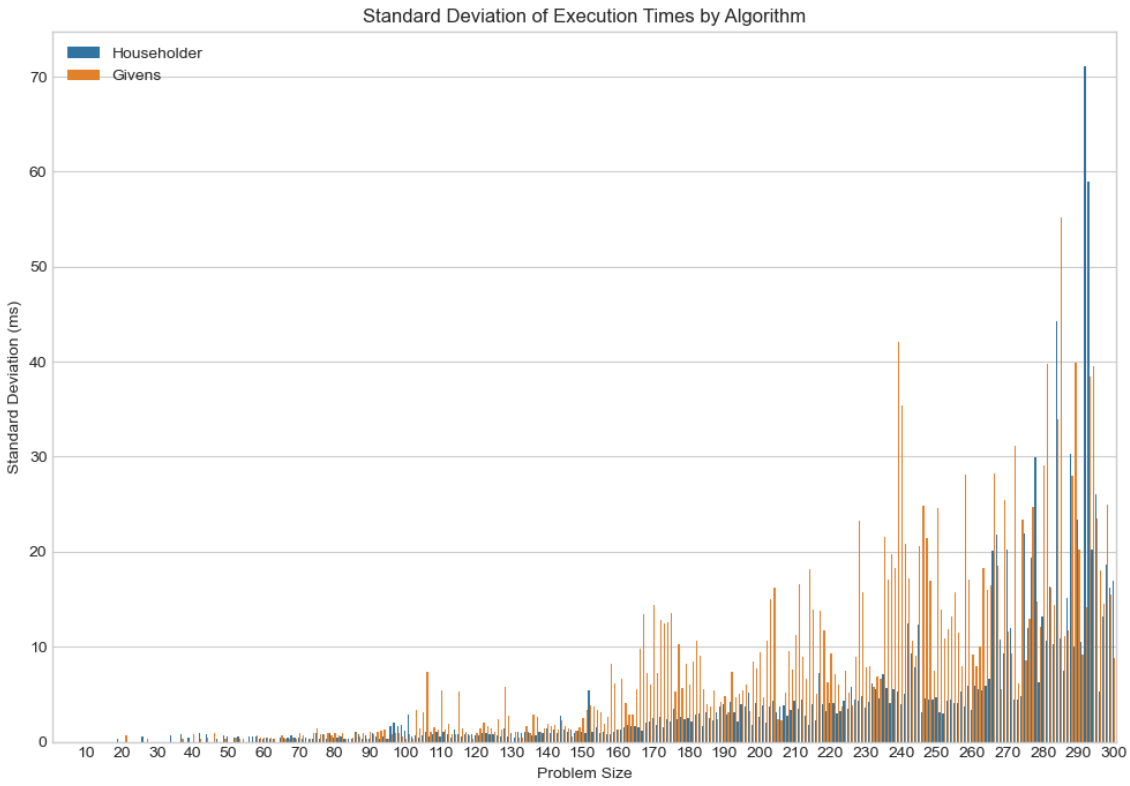

Figure 6: Standard deviation of execution times by algorithm

6 Conclusion

In conclusion, all objectives of the programming project have been successfully achieved. The result of the accomplished work is `LinAlgTools`, a C++ library, offering matrix arithmetic and decomposition algorithms. Rigorous testing and performance benchmarking confirm its reliability, while the intentionally clean, well-structured source, code, publicly available on github, makes it accessible to anyone with some knowledge of C++.

References

[1] Joseph W. H. Liu Alan George. Householder reflections versus givens rotations in sparse orthogonal decomposition. URL: https://www.sciencedirect.com/science/article/pii/002437958790111X, 2002.

[2] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen. *LAPACK Users' Guide*. Society for Industrial and Applied Mathematics, Philadelphia, PA, third edition, 1999.

[3] Charles F. Van Loan Gene H. Golub. Matrix computations, 4th edition. URL: https://doi.org/10.56021/9781421407944, 2013.

[4] Google. Googletest. URL: https://github.com/google/googletest, 1.15.2.

[5] Gaël Guennebaud, Benoît Jacob, et al. Eigen v3. URL: http://eigen.tuxfamily.org, 2010.

[6] Millman K.J. van der Walt S.J. et al Harris, C.R. Numpy. URL: https://github.com/numpy/numpy, 2025.

[7] Davis E. King. Dlib-ml: A machine learning toolkit. *Journal of Machine Learning Research*, 10:1755–1758, 2009.

[8] Conrad Sanderson and Ryan Curtin. Armadillo. URL: https://arma.sourceforge.net/armadillo_iccae_2025.pdf, 2025.

[9] Zinkin Zakhar. LinAlgTools. URL: https://github.com/Avgustineiw/LinAlgTools.